Chapter 1
Visualization Assessment of Tendency in Clustering – 58K × 58K in 60 seconds

Scott M Phillips[1] 0009-0003-9169-7519
Kelly Cohen PhD[2] 0000-0002-8655-1465

Abstract Visualization Assessment of Tendency (VAT) in Clustering has been a technique for the identification of data clusters since 2002. Although a proven technique, it has suffered from the curse of dimensionality, leading to limited performance for datasets with more than 5000 elements. The core of the model is based on Prim's algorithm, a greedy Minimum Spanning Tree search that uses Bubble Sort at its core. By replacing Bubble Sort with Merge Sort using a priority queue, the updated "mergeVAT" algorithm complexity changes from $O(N) = N^3$ to $O(N) = N^2 \log N$. For datasets with 1,000 elements, this leads to a speed up of 30×. In addition, by identifying the permutation loops and bit-masking, it is now possible to compute the full VAT of a 58000 element dataset in place while requiring only $O(N) = \frac{N^2}{16}$ additional working memory in less than 2 minutes.

1.1 Background

1.1.1 VAT

VAT is a method for the visual identification of clusters in arbitrary dimension data sets. It converts the arbitrary dimensions into a 2D dissimilarity matrix by using a norm function. Most commonly, this is the L2-norm, also known as the euclidean norm. However, other norms may be used depending on the application and the desired measure of dissimilarity. This converts the $N \times K$ matrix of samples into a $N \times N$ matrix of pairwise distances, commonly called the dissimilarity matrix D. The core of the method is based upon a minimum spanning tree of the columns of the dissimilarity matrix D. More specifically, this method is very similar to Prim's

Scott Phillips, Kelly Cohen
University of Cincinnati, Cincinnati OH 45221, USA,
e-mail: phillis8@mail.uc.edu, cohenky@ucmail.uc.edu

algorithm – a greedy algorithm for computing the minimum spanning tree.

Since the original development of the VAT method [4], numerous alternate and custom implementations have been developed. Broadly speaking, these have attempted to address the issue of performance in Prim's algorithm. By enforcing other constraints on order, distance metric, or output, the methods improve the performance [5] [6]. Some methods also focus on specific implementation enhancements such as memory access patterns [8]. At the end of the day, the key complexity of Prim's method, often referenced in the literature as $O(N) = N^2$, remains unchanged [7] [9].

1.1.2 Sorting

1.1.2.1 Bubble Sort

The most basic implementation of Prim's algorithm is broadly similar to a two-dimensional bubble sort. Bubble sort [2] being one of the most simple sorting algorithms available. For small arrays, the simplicity of development outweighs the lower performance relative to other methods. It also has optimal memory utilization, $O(N) = 1$. In contrast, its computational complexity in both the average and the worst case is $O(N) = N^2$. As a result, its performance on large random sequences is particularly poor [3]. Other sorting methods exist which address this issue by balancing a trade-off of memory consumption and computational complexity.

1.1.2.2 Merge Sort

In this particular instance, the chosen method is Merge Sort. Merge sort is the fastest possible sorting algorithm [1]. Its computational complexity is $O(N) = N \log N$. Its additional memory consumption is $O(N) = N$. It is important to note that this is the case for sorting a one-dimensional array. For a two-dimensional square array where the sorting is applied on a per-column (or per-row) basis, the sorting algorithm multiplies another factor N, leading to Bubble Sort complexity as $O(N) = N^3$ and Merge Sort complexity as $O(N) = N^2 \log N$.

1.2 Key Insight – mergeVAT

While Prim's algorithm is historically based upon a bubble sort methodology by searching the adjacency matrix, there exists an alternate formulation that utilizes a priority queue to increase the computational efficiency [11]. This priority queue allows extraction of the lowest-value entry in the array as an $O(N) = 1$ operation

since the lowest value (highest priority) entry is always first in the queue. The key insight in this paper is the application of that priority queue based method to VAT. While the adaptation of Prim's algorithm to a priority queue method is not particularly difficult, it is important to note the difference between output of a sequence of segment connections as needed by a minimum spanning tree versus the unique sequence of the permutation array as required by VAT. It is also important to anchor the methods so that it preserves the greedy approach as expected by VAT in order to create optimal visual clusters.

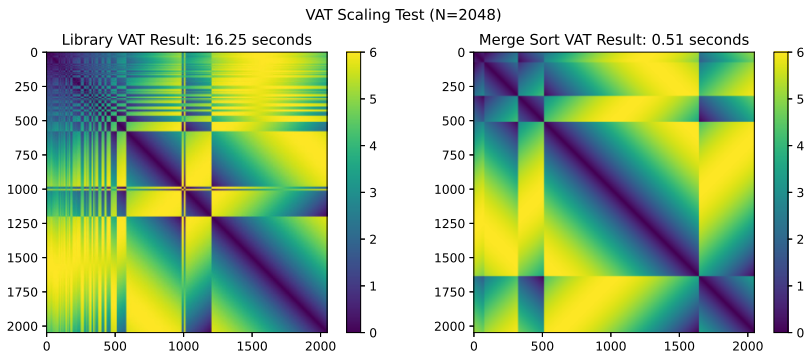

Fig. 1.1 VAT vs mergeVAT Output comparison with Block Swapping Issue

By instrumenting the existing VAT method implementations [4], it was possible to identify the mapping sequence between the minimum spanning tree segments and the permutation array output. From there, it was possible to identify the key point in the priority queue based Prim's algorithm where in the unique sequence of the permutation array could be extracted directly. Prior to this insight, the merge sort method required a second pass through the minimum spanning tree segment sequence to order the unique output. This had the side effect of additional memory consumption because Python `sets` do not preserve insertion order. In fact, Python `set()` will order integers from smallest to largest. This explicit ordering would make the output useless since a permutation of the ordered sequence is required.

1.3 Key Insight – Permutation Loops

The second key in sight of this paper was to identify that the permutation array P creates a sequence of loops with the original column sequence $Q \in [1,N]$. These permutation loops can range in length from 1 to N. The length-1 (singleton) loops implying $P_i = Q_i$. A loop of length N implies there is exactly one permutation loop for the entire array. Because each loop is independent of the other loops that may

exist, a permutation sequence can follow one loop from one point to another until it returns to the starting point which can be anywhere along the loop.

This realization drove an alternate implementation of the output dissimilarity matrix D'; this alternate implementation starts at $D_{1,1}$ and iteratively applies the permutation array to the indices. An additional bit masking array B is used to keep track of which elements have been already permuted. Once the permutation sequence returns to the starting point (initially $1,1$), the starting index is moved to $D_{1,2}$ and the process is repeated. If any element has already been permuted, it is skipped. Because each loop is completed in sequence, only one temporary variable is required to hold $D_{i,j}$ and apply it to $p^{-1}(i), p^{-1}(j)$. Because the dissimilarity matrix D is symmetric, the symmetric points on the principal diagonal are always updated concurrently. This also allows the permutation bit mask B to be triangular with a size order of

$$O(N) = \frac{N^2 + N}{2 \times 8} \tag{1.1}$$

The additional divisor of 8 is due to eight bits in a byte. In the interest of simplicity and avoiding odd memory stride issues, the author chose to bit-mask each row individually, which gives a theoretical minimum wasted space of up to seven bits (plus any stride allocation used by the memory allocator). This wasted space was deemed reasonable in the current era of low cost RAM :).

A further space savings was driven by a request from the authors' colleagues. It was asked if it would be possible to analyze a data set with 136K records and output simply the permutation sequence. The author noted that this dataset would require approximately 148GB of RAM for the dissimilarity matrix alone. However, upon further thought, it was realized that the dissimilarity matrix can always be computed on the fly and only across the relevant elements. In so doing, the additional working space would go from 148 GB to only a few GB over the raw dataset. This would be at the cost of on-the-fly recomputation of dissimilarity matrix elements that had already been computed. This would prove to be a large CPU trade-off in exchange for an order of magnitude reduction in memory.

1.4 Scaling Tests

Upon achieving this sequenced output directly from the "mergeVAT" method, scaling tests were performed to identify how much faster the method would prove in practice. The literature seems to agree that the maximum size of an effective VAT is around 5,000 elements. Utilizing the author's computer as a benchmark, a 5,000 element VAT using a bubble sort methodology requires about 2 minutes to run. This mark of 2 minutes as a maximum acceptable runtime; larger and larger data sets were explored to identify how many elements were possible to order in less than 2 minutes.

1.4.1 Synthetic Data

In the lead up to this analysis of 58,000 element data set the author used the same methodology and synthetic data set used in his previous paper at NAFIPS 2025. This sequence of circles distributed around a larger circle was used because it creates a synthetic data set of arbitrary size with known cluster sizes that are distinctly separated or conjoined if desired. Additionally, because of the interest in the applications of VAT to TSP style problems, this synthetic data set has a trivial analytic approximate solution to the TSP. The formula for this is shown below. This synthetic data set was used to show the difference in scaling performance between the traditional VAT method and the merge VAT method. The differences are shown in 1.1. There are complexities and outside factors on the timing, so the theoretically perfect scaling is shown for N^2, $N^2 \log N$, and N^3. The smaller data set samples are more influenced by external factors such as other processes and the individual random layout provided. Therefore, the larger sample sizes show more clearly the performance advantages of this merge VAT algorithm.

1.4.2 UCI-ML NASA Dataset with 58K points

An important part of testing with real world data was to ensure that the output data set had some explicit clustering. Most of the common cluster example data sets are very small and so the key performance gains as realized by this alternate implementation would not be sufficiently evident. A data set from the University of California Irvine machine learning repository [12] was used. This data set, provided by NASA, has about 58,000 elements of which approximately 80% belong to a single cluster. The dataset is a time-series relating 7 input parameters to an output class condition. While this uneven distribution in clusters might be problematic in a machine learning training scenario, it is ideal for large-scale analysis of a visual method because this key 80% structure will be visible in the output data set even if the output image is scaled down.

This 58,000 element data set could be ordered on the author's computer, a 14th generation Intel core 9, 14980HX with 96GB of RAM in about 2 minutes utilizing approximately 40 GB of RAM. It is important to note that this relatively low memory consumption is based upon an additional insight in the form of in-place VAT operations which were described above. Utilizing a direct method that requires copying from the source dissimilarity matrix D to the VAT ordered dissimilarity matrix D' uses a maximum of approximately 80 GB of RAM, while having a runtime of 55-60 seconds. An important note about this is that the dissimilarity matrix D for this test case has 3.36 billion elements in it. All tests were performed with 64-bit floating point numbers, leading to a minimum of 24 gigabytes of memory being consumed by the dissimilarity matrix alone. More detailed memory consumption

analysis could be performed, but it is also subject to external factors that are more difficult to control. The image shown in 1.4 is the output of this VAT operation scaled down by a factor of 100 in each dimension, making the output image 10,000 times smaller in area. This was done to facilitate easy sharing of the image, because the source image even in 8-bit grayscale compressed PNG was still over 400 MB.

1.5 Future Work and Other Notes

1.5.1 Stepwise Refinement

One noteworthy behavior that bears further analysis is shown in 1.1. Presumably the random permutation of structured data, the VAT ordering will appear to have a few segments "flipped" from the optimal order. This is difficult to identify in the initial data, it is immediately evident in the output VAT. This situation likely relates to The TSP situation where the route takes a less than optimal jump across the space. This phenomenon is present in both the original VAT and the mergeVAT. Further work is needed to develop an identification method for this and potentially apply a corrective action step post VAT. Some initial work using a 2-OPT TSP refinement method was applied, and it appears to improve the image quality, as shown in [1.5]

1.5.2 Conclusion

Replacing the common bubble-sort based method for Prim's algorithm with a merge-sort based method provides a substantial performance gain, especially as the number of elements in the matrix grows larger. This technique is broadly re-applicable to the other formulations of *VAT, since it alters the kernel of the minimum spanning tree calculation versus the associated calculations for dissimilarity. The loop-following permutation method is another useful development, since it provides a direct tradeoff between CPU and memory utilization, which allows larger dissimilarity matrices to be ordered, especially with the scaling capabilities provided by the mergeVAT method.

References

1. Katajainen, J.; Pasanen, T.; Teuhola, J.: Practical In-place Mergesort. Nordic Journal of Computing **3**(1), 27-40 (1996)
2. Friend, E.: Sorting on Electronic Computer Systems. Journal of the ACM **3**(3), 134-168 (1956)
3. Astrachan, O.: Bubble Sort: An Archeological Algorithm Analysis. ACM SIGCSE Bulletin **35**(1), 1-5 (2003)

4. Bezdek, J.; Hathaway, R.: VAT: a tool for visual assessment of (cluster) tendency. In: Proceedings of the 2002 International Joint Conference on Neural Networks, pp. 2225-2230 vol.3, Honolulu, HI, USA, (2002)
5. Bezdek, J.; Havens, T.: An Efficient Formulation of the Improved Visual Assessment of Cluster Tendency (iVAT) Algorithm. https://pages.mtu.edu/~thavens/papers/TKDE-2010-06-0336.R1_Havens.pdf Last Accessed 2025-11-19
6. Rathore, P.; Bezdek, J. Santi, P.; Ratti, C.: ConiVAT: Cluster Tendency Asessment and clustering with Partial Background Knowledge. IEEE Transactions On Knowledge and Data Engineering https://arxiv.org/pdf/2008.09570 (2008)
7. Havens, T.; Bezdek, J.; Keller, J.; Popescu, M.; Huband, J.: Is VAT Really Single Linkage in Disguise. Ann Math Artificial Intelligence https://pages.mtu.edu/~thavens/papers/AMAI_2009_55_Havens.pdf (2009)
8. Avinash, MSR; Lachheb, I.: Fast-VAT: Accelerating Cluster Tendency Visualization using Cython and Numba. https://arxiv.org/pdf/2507.15904 (2025)
9. Hathaway, R.; Bezdek, J.; Huband, J.: Scalable Visual Assessment of Cluster Tendency For Large Data Sets. Pattern Recognition **39**7, 1315-1324 (2006)
10. Komarasamy, G.; Bezdek, J.; Wahi, A.: Finding the Number of Clusters using Visual Validation VAT Algorithm. Interational Journal of Engineering and Technology, pp. 3951-3957 vol.5, https://pages.mtu.edu/~thavens/papers/AMAI_2009_55_Havens.pdf (2009)
11. Pettie, S.; Ramachandran, V.: An Optimal Minimum Spanning Tree Algorithm. Journal of the ACM, Vol 49, No. 1, pp 16-34, (2002)
12. ”Statlog (Shuttle),” UCI Machine Learning Repository, [Online]. Available: https://doi.org/10.24432/C5WS31.

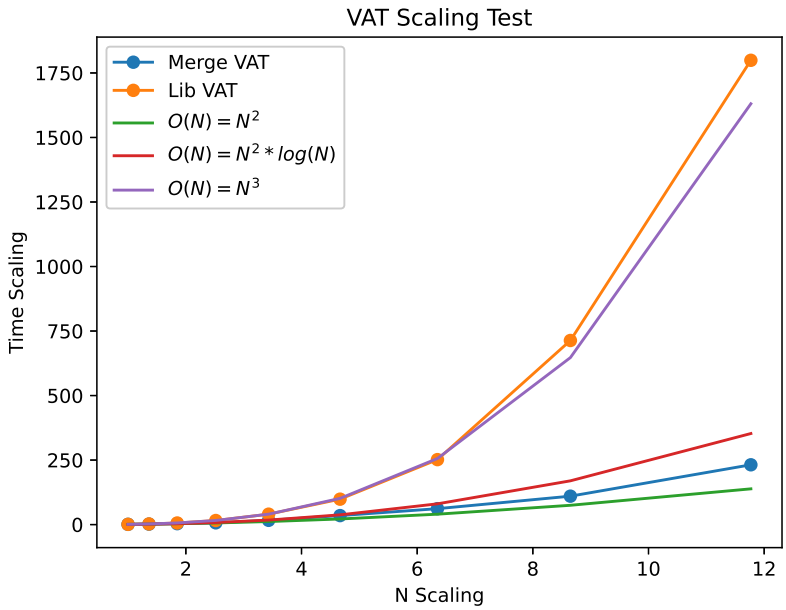

Fig. 1.2 VAT Implementation Scaling Comparisons

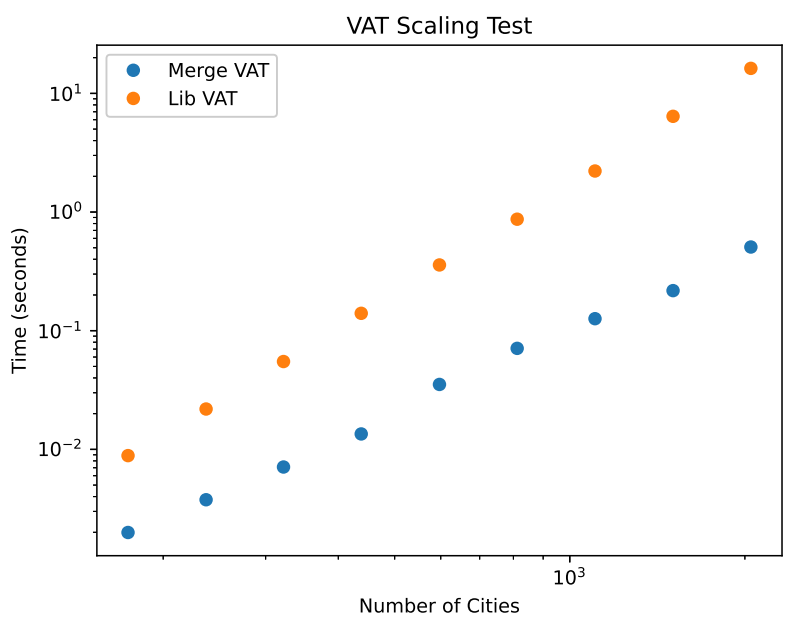

Fig. 1.3 VAT Implementation Time Scaling Comparison – Core i7 1185G7 @ 3.00 GHz

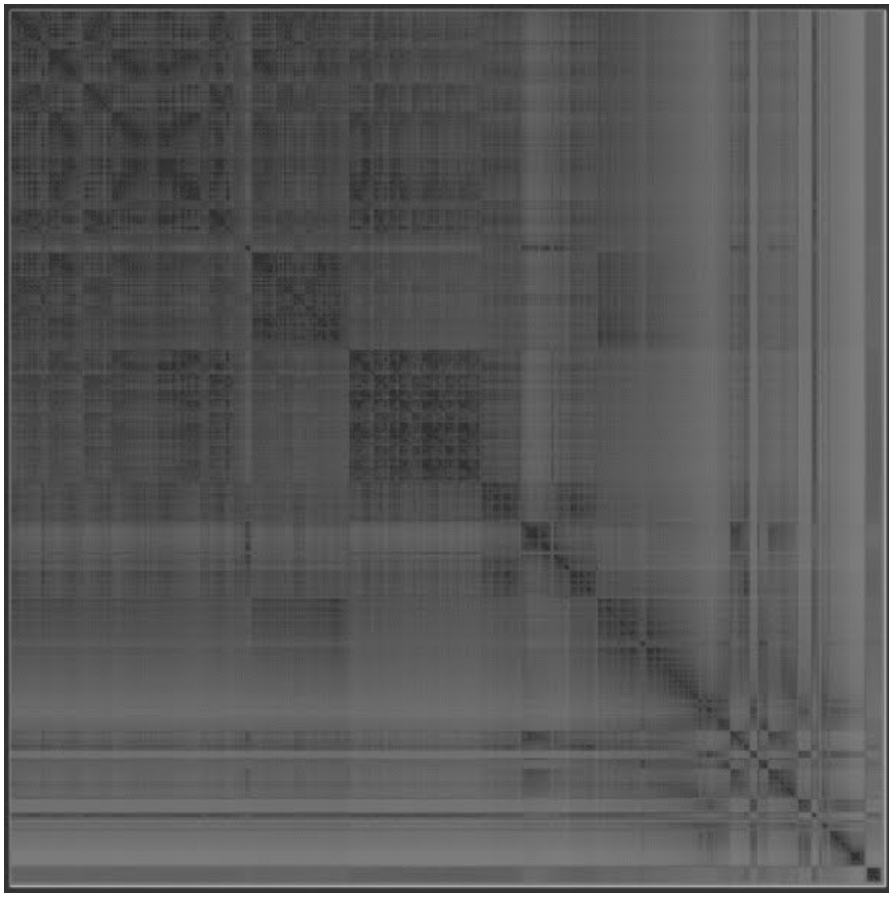

Fig. 1.4 UCI ML NASA Dataset – 1% scale

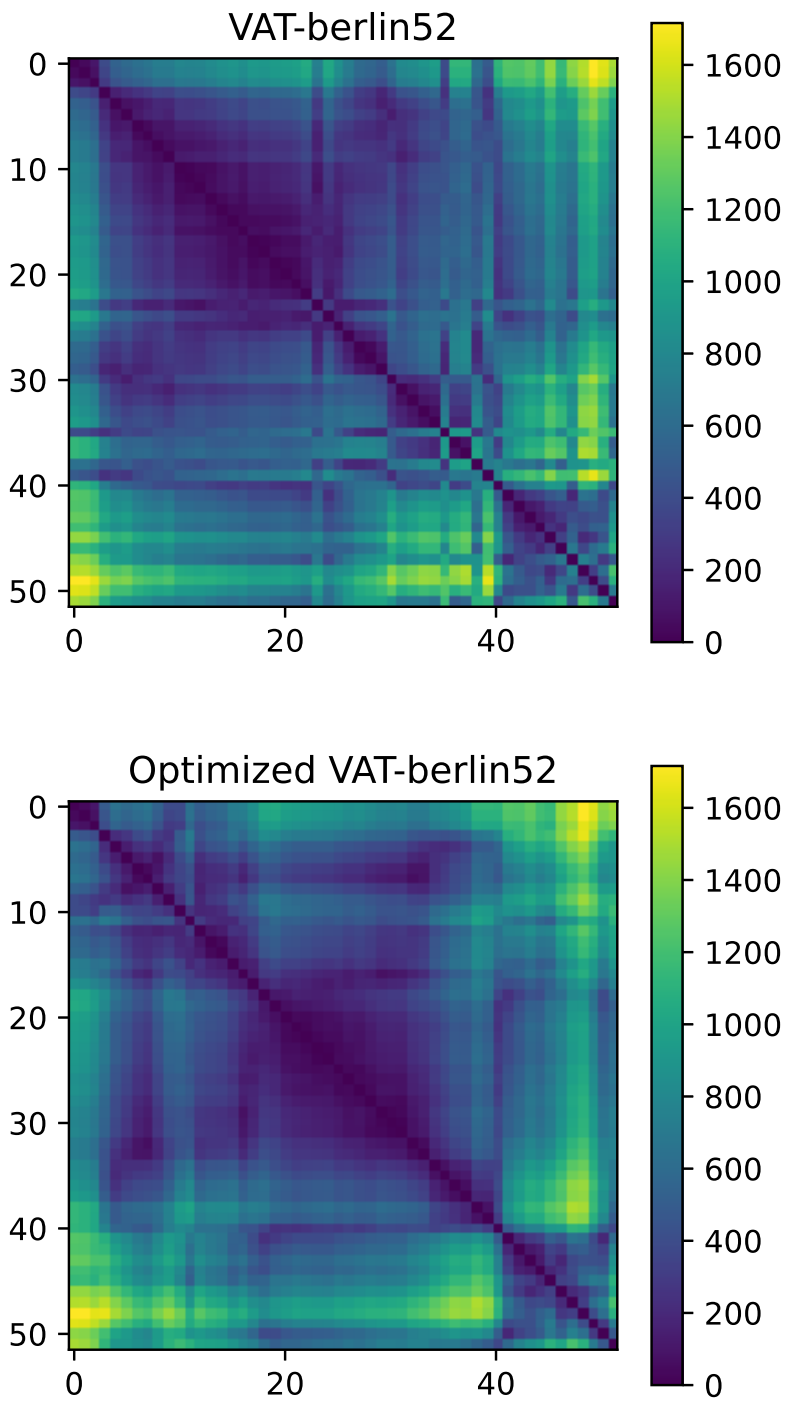

Fig. 1.5 VAT vs Optimized VAT – Berlin52